JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.logging

Interface LoggingMXBean

`public interface` **`LoggingMXBean`**

The management interface for the logging facility. It is recommended to use the `PlatformLoggingMXBean` management interface that implements all attributes defined in this `LoggingMXBean`. The `ManagementFactory.getPlatformMXBean` method can be used to obtain the `PlatformLoggingMXBean` object representing the management interface for logging.

There is a single global instance of the `LoggingMXBean`. This instance is an `MXBean` that can be obtained by calling the `LogManager.getLoggingMXBean()` method or from the platform `MBeanServer`.

The `ObjectName` that uniquely identifies the management interface for logging within the `MBeanServer` is:

```
java.util.logging:type=Logging
```

The instance registered in the platform `MBeanServer` is also a `PlatformLoggingMXBean`.

Since:

1.5

See Also:

`PlatformLoggingMXBean`

Method Summary

Abstract Methods

Modifier and Type	Method	Description
String	getLoggerLevel(String loggerName)	Gets the name of the log level associated with the specified logger.
List<String>	getLoggerNames()	Returns the list of currently registered logger names.
String	getParentLoggerName(String loggerName)	Returns the name of the parent for the specified logger.
void	setLoggerLevel(String loggerName, String levelName)	Sets the specified logger to the specified new level.

Method Detail

getLoggerNames

```
List<String> getLoggerNames()
```

Returns the list of currently registered logger names. This method calls `LogManager.getLoggerNames()` and returns a list of the logger names.

Returns:

A list of String each of which is a currently registered Logger name.

getLoggerLevel

```
String getLoggerLevel(String loggerName)
```

Gets the name of the log level associated with the specified logger. If the specified logger does not exist, `null` is returned. This method first finds the logger of the given name and then returns the name of the log level by calling:

```
Logger.getLevel().getName();
```

If the `Level` of the specified logger is `null`, which means that this logger's effective level is inherited from its parent, an empty string will be returned.

Parameters:

`loggerName` - The name of the Logger to be retrieved.

Returns:

The name of the log level of the specified logger; or an empty string if the log level of the specified logger is null. If the specified logger does not exist, null is returned.

See Also:

`Logger.getLevel()`

setLoggerLevel

```
void setLoggerLevel(String loggerName,
                    String levelName)
```

Sets the specified logger to the specified new level. If the `levelName` is not `null`, the level of the specified logger is set to the parsed `Level` matching the `levelName`. If the `levelName` is `null`,
the level of the specified logger is set to `null` and the effective level of the logger is inherited from its nearest ancestor with a specific (non-null) level value.

Parameters:

`loggerName` - The name of the Logger to be set. Must be non-null.

`levelName` - The name of the level to set on the specified logger, or null if setting the level to inherit from its nearest ancestor.

Throws:

`IllegalArgumentException` - if the specified logger does not exist, or levelName is not a valid level name.

`SecurityException` - if a security manager exists and if the caller does not have LoggingPermission("control").

See Also:

Logger.setLevel(java.util.logging.Level)

getParentLoggerName

`String getParentLoggerName(String loggerName)`

Returns the name of the parent for the specified logger. If the specified logger does not exist, `null` is returned. If the specified logger is the root `Logger` in the namespace, the result will be an empty string.

Parameters:

`loggerName - The name of a Logger.`

Returns:

`the name of the nearest existing parent logger; an empty string if the specified logger is the root logger. If the specified logger does not exist, null is returned.`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.